

# Performance Testing Seminar Report

---

**Course:** Software Testing (CS423/CSC15003) — FIT, HCMUS **Topic:** T05 — Performance Testing **System Under Test (SUT):** EShop (Node.js + Express + SQLite) **Group:** Team 09

---

## 1. Abstract

This report presents a performance testing study of **EShop**, a Node.js/SQLite e-commerce backend, carried out as part of the T05 — Performance Testing seminar. The study defines a mixed workload model covering product browsing, search, cart, and checkout, implements that model independently in both Apache JMeter and k6, and evaluates the system under a baseline load (50 virtual users) and a spike load (50 to 500 virtual users within 30 seconds). A large language model (LLM) was used to draft a "realistic shopping" load script from a captured HTTP log, which the team then audited and corrected before use — an AI-assisted supplement to the two traditional tools, not a replacement for them.

The report covers the conceptual background of performance testing, the system under test, the workload model, the experimental methodology, the AI-assisted scripting workflow, the resulting measurements, and a comparative evaluation of the two tools.

**Key outcomes:** At baseline load, EShop passed both latency targets with a wide margin (p95 of 2.54 ms in k6 and 5 ms in JMeter), but failed the error-rate SLO: roughly 40% of requests returned authentication errors in both tools, consistent with EShop's login-lockout behavior interacting with the shared test accounts. Under the spike load, the latency targets were breached as well (p95 roughly 1.45 s and p99 above 2.3 s in both tools) while the authentication-related error rate persisted at roughly 40%. The overall verdict is an SLO failure at both load levels, driven by the error-rate ceiling at baseline and by both error rate and latency under the spike. For the requests that did succeed, latency under the spike grew far faster than load did, a pattern consistent with write contention on SQLite's single database connection, which every cart and checkout request must pass through.

---

## 2. Team Information

### 2.1 Course & Seminar Information

| Field         | Value                                                                   |
|---------------|-------------------------------------------------------------------------|
| Course        | Software Testing (CS423/CSC15003), FIT – University of Science, VNU-HCM |
| Supervisors   | Dr. Lâm Quang Vũ, MSc. Trương Phước Lộc                                 |
| Seminar Topic | T05 – Performance Testing                                               |
| Group         | Team 09                                                                 |

### 2.2 Team Members

| Student ID | Full Name |
|------------|-----------|
|------------|-----------|

| Student ID | Full Name             |
|------------|-----------------------|
| 23127006   | Trần Nguyễn Khải Luân |
| 23127128   | Nguyễn Thành Tiến     |
| 23127179   | Nguyễn Bảo Duy        |
| 23127184   | Lê Phạm Kiều Duyên    |

### 3. Introduction

Modern web applications must remain responsive and stable as user load grows. Functional testing verifies that a system behaves correctly for a single user or a small number of requests, but it does not reveal how a system behaves under many concurrent users, under a sudden traffic spike, or when a shared resource (such as a database connection) becomes a bottleneck. Performance testing addresses this gap by measuring response time, throughput, and error behavior under controlled, repeatable load conditions.

This seminar uses **EShop**, a Node.js/Express backend with a SQLite database, as the system under test. EShop is a small e-commerce application exposing REST endpoints for product browsing, search, cart management, and checkout, secured with a JWT-based login flow.

### 4. Performance Testing Fundamentals

#### 4.1 What is Performance Testing?

Performance testing is a non-functional testing discipline that evaluates how a system behaves with respect to responsiveness and stability under a given workload. Unlike functional testing, which asks "does the system produce the correct output," performance testing asks "how fast, how consistent, and how far can the system go before it degrades."

#### 4.2 Types of Performance Testing

| Type                     | Purpose                                                                                                                          |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Load testing             | Verify behavior under an expected, realistic concurrent user load (this seminar's "baseline" scenario).                          |
| Stress testing           | Push load beyond expected capacity to find the breaking point.                                                                   |
| Spike testing            | Apply a sudden, sharp increase in load and observe behavior and recovery (this seminar's "spike" scenario, 50 → 500 VU in 30 s). |
| Soak / endurance testing | Sustain a moderate load over a long duration to detect memory leaks or resource exhaustion.                                      |
| Scalability testing      | Determine how throughput changes as load or resources are incrementally increased.                                               |

This seminar focuses on **load testing** (baseline) and **spike testing**. Soak, stress-to-failure, and scalability testing are out of scope; see Section 13 (Discussion).

### 4.3 Performance Metrics

- **Response time / latency**, typically reported as percentiles (p50, p95, p99) rather than an average, because averages hide tail latency that affects a meaningful fraction of users.
  - **Throughput**, the number of requests (or transactions) completed per unit time.
  - **Error rate**, the percentage of requests that fail (non-2xx status, timeout, or connection error).
  - **Resource utilization** (CPU, memory, I/O, database connections), when instrumentation is available, to help localize a bottleneck.
- 

## 5. EShop System & Testing Scope

### 5.1 EShop Overview

EShop is a small e-commerce web application built on a Node.js/Express backend with a SQLite database. It provides the core business functions of an online store: product browsing and search, user authentication, cart management, and checkout. These functions are exposed through a REST API.

### 5.2 System Architecture

- **Backend**: a single-process Express.js (Node.js) application.
- **Database**: SQLite, used to persist product, user, and order data.
- **Authentication**: session-based login using JWT tokens.
- **Rate limiting**: the API applies a request-rate limit per client IP address.

### 5.3 Testing Scope

The performance test targets four core user workflows:

- **Browse** — viewing the product listing.
- **Search** — searching for products by keyword.
- **Add to Cart** — adding a product to an authenticated user's cart.
- **Checkout** — submitting an authenticated order.

**Environment Setup Note:** To ensure that the high-load scenario (Spike Test up to 500 VUs) accurately reflects the system's processing limits rather than being blocked early at the network layer, the EShop backend environment is launched with the `LOADTEST=1` flag. This configuration actively bypasses the request rate limiter (200 req/15min), isolating the system and focusing the performance measurement on the underlying business logic and database layer.

**Out of scope:** administrative endpoints, non-HTTP protocols, and browser/UI-level testing — this is an API-level performance test.

---

## 6. Tool Selection

The team selected **k6** as the primary tool, **Apache JMeter** as the secondary tool, and an LLM-assisted (ChatGPT/Claude) scripting workflow as the AI-assisted direction.

- **k6** — JavaScript scripting lets a human and an LLM draft and review scripts in the same language, and its built-in thresholds fit repeatable baseline/spike runs.
- **Apache JMeter** — a standard, GUI-supported tool that is easier to demonstrate to a non-technical audience; measured runs use its non-GUI mode.
- **AI-assisted scripting** — included because the seminar requires pairing a traditional tool with an AI direction, giving the team a real script to audit (Section 10).

Neither tool is assumed superior; Section 12 compares them using the results in Section 11.

---

## 7. Workload Model for EShop

### 7.1 Rationale

The workload model targets four core EShop actions — browse, search, add-to-cart, and checkout — using a browse-heavy, checkout-light distribution. This reflects typical e-commerce usage: most sessions browse or search, and only a minority complete a purchase.

### 7.2 Workload Distribution

| Action      | Weight |
|-------------|--------|
| Browse      | 40%    |
| Search      | 35%    |
| Add to Cart | 20%    |
| Checkout    | 5%     |

### 7.3 Think Time

Think time between actions is randomized between 1 and 3 seconds to approximate realistic user pacing rather than back-to-back requests.

### 7.4 Baseline Workload

A steady-state load: ramp from 0 to 50 virtual users over 1 minute, hold 50 VU for 5 minutes, then ramp down over 1 minute.

### 7.5 Spike Workload

A sudden increase: start at 50 VU, ramp to 500 VU over 30 seconds, hold for 2 minutes, then ramp back down to 50 VU over 1 minute.

### 7.6 SLO Definition

- p95 response time < 500 ms
- p99 response time < 1000 ms

- Error rate < 1%

---

## 8. Experimental Methodology

### 8.1 Test Environment

| Item             | Value                      |
|------------------|----------------------------|
| CPU              | Apple M1 (8 cores)         |
| RAM              | 16 GB                      |
| Operating System | macOS 26.5.1 (Build 25F80) |
| Node.js          | v24.4.1                    |
| Apache JMeter    | 5.6.3                      |
| k6               | v2.1.0                     |

The backend under test and the load generator ran on the same machine; this co-location is a known threat to validity discussed in Section 13.

### 8.2 Baseline Test

A steady-state load test at 50 virtual users, using the workload model in Section 7, executed identically in both k6 and Apache JMeter so the two tools are compared under the same conditions.

### 8.3 Spike Test

A sudden increase from 50 to 500 virtual users within 30 seconds, using the same workload model, to observe how the system behaves under a sharp traffic surge and whether it recovers once load returns to baseline.

### 8.4 Collected Metrics

- p50 latency
- p95 latency
- p99 latency
- Throughput
- Error rate

---

## 9. Demo Script

This section describes only the 10-minute **Live Demo** slot of the 45-minute seminar (Pitch 10 min → Live Demo 10 min → Audience Activity 20 min → Q&A 5 min). The Pitch, the Audience Activity, and the Q&A are outside the scope of this section.

### 9.1 Demo Objective

The live demonstration shows the audience how the workload model defined in Section 7 is implemented and evaluated using Apache JMeter and k6, and how an AI-assisted script fits into that same workflow as a

supporting artifact rather than a separate demonstration. It grounds the concepts introduced during the Pitch in an observable run of the system, and establishes the shared vocabulary — workload mix, baseline, spike, SLO — that the audience needs before designing their own workload model in the Activity that follows.

## 9.2 Demo Preparation

The following is confirmed ready before the seminar begins; none of it is set up live on stage:

- EShop is already running.
- The JMeter and k6 scripts for the workload model (Section 7) are already implemented and validated.
- The AI-generated script draft is available, alongside the corrected version that will actually run.
- A completed run of each scenario is captured in advance as a fallback.

Because the complete baseline and spike scenarios exceed the available demonstration time, completed measurement results are prepared in advance as a fallback. The live demonstration focuses on illustrating the execution workflow rather than waiting for every scenario to finish.

## 9.3 Live Demo Timeline

| Time          | Demonstration                                                                                                                                                                        | Key Message                                                                                        |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| 0:00–<br>1:00 | Frame the demo: one EShop workload, one testing objective, evaluated with two tools and one AI-assisted script.                                                                      | One testing objective, realized through two tools and an AI-assisted script — not a tool showcase. |
| 1:00–<br>2:30 | Walk through the workload model — action mix and think time (Section 7) — before opening either tool.                                                                                | Workload design comes before tool execution.                                                       |
| 2:30–<br>4:00 | Demonstrate the baseline workload using JMeter: open the GUI test plan to explain how the workload is represented, then show the results from the completed non-GUI measurement run. | JMeter expresses the same workload as a GUI-authored, non-GUI-measured plan.                       |
| 4:00–<br>5:30 | Demonstrate the ongoing baseline execution in k6; narrate the streaming summary and threshold results as they appear.                                                                | The identical workload, re-expressed as a JavaScript script — same objective, different tool.      |
| 5:30–<br>6:30 | Trigger the spike scenario in k6 to illustrate the rapid increase in load; the completed spike results are presented immediately afterward.                                          | The same workload model also drives the spike test; no separate script is written for it.          |
| 6:30–<br>8:00 | Show the AI-generated performance test script and the log it was drafted from, then walk through why the version actually executed differs after human review.                       | AI drafts a starting point; the version that runs is the one a human reviewed and corrected.       |
| 8:00–<br>9:00 | Place the JMeter and k6 summaries — both captured in full beforehand — side by side.                                                                                                 | Two tools, one workload, comparable evidence — not a competition between tools.                    |

| Time       | Demonstration                                                 | Key Message                                                                                   |
|------------|---------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| 9:00–10:00 | State the key takeaway and hand off to the Audience Activity. | Workload modeling, not tool choice, decides whether a performance test result can be trusted. |

9.4 Expected Learning Outcomes

By the end of the Live Demo, the audience should understand that:

- Workload modeling is defined before any tool is touched, and both tool implementations are expected to reproduce it exactly.
- Running the same workload consistently across tools — not raw tool speed — is what makes a JMeter-vs-k6 comparison meaningful.
- Apache JMeter and k6 have different strengths (GUI-based explainability versus code-based, CI-friendly scripting); neither is presented as universally better.
- AI can accelerate first-draft script generation, but engineering judgment remains essential before any generated script is used in performance testing.

These outcomes lead directly into the Audience Activity: now that the audience has seen how a workload is executed, they will design one themselves.

10. AI-Augmented Performance Testing

10.1 How AI Was Used

An LLM was used to draft a "realistic shopping" k6 script from a captured HTTP log, covering a single shopping session (browse → search → add-to-cart → checkout). The team then reviewed and corrected the draft before using it.

10.2 Key Findings

| Issue                                                | Human Fix                                      |
|------------------------------------------------------|------------------------------------------------|
| Endpoint in the draft does not exist in the system   | Corrected to the real search endpoint          |
| Missing authentication before cart/checkout requests | Added a login step to obtain a session token   |
| No think time between actions                        | Adjusted to a randomized 1–3 second pause      |
| Hard-coded product/user identifiers                  | Parameterized from the seeded catalog          |
| Fixed action sequence, not matching the workload mix | Flagged for reconciliation with Section 7.2    |
| Weak status-code assertions                          | Tightened to assert an exact successful status |

10.3 Human Review Process

Every AI-generated script was reviewed against the real system before use. The endpoint and authentication fixes above were verified and applied directly. The full artifact-by-artifact review record, including verdicts and

the verification source used for each correction, is documented in the attached AI Audit Report ([AI-02]).

## 11. Experimental Results & Analysis

Metrics are measured against the thresholds defined in Section 7.6 and Section 8.4.

### 11.1 Baseline Results

| Metric        | k6           | JMeter       | SLO Threshold | Result      |
|---------------|--------------|--------------|---------------|-------------|
| p50 latency   | 1.23 ms      | 1.00 ms      | -             | Pass        |
| p95 latency   | 2.54 ms      | 5.00 ms      | < 500 ms      | Pass        |
| p99 latency   | 66.34 ms     | 73.00 ms     | < 1000 ms     | Pass        |
| Error rate    | 39.76%       | 39.89%       | < 1.0%        | Fail (Auth) |
| SLO pass/fail | Fail (error) | Fail (error) | -             | Fail        |

At 50 VU, both tools recorded latency far below the targets: p95 stayed in the single-digit milliseconds and p99 under 100 ms, against 500 ms and 1000 ms thresholds. The error rate, however, failed its SLO by a wide margin: roughly 40% of requests returned errors in both tools, labeled as authentication failures. This matches EShop's documented login-lockout behavior (an account is locked after two failed attempts), which the shared test-account pool triggered under concurrency. The two tools agree closely on both figures, which gives confidence the numbers reflect EShop's actual behavior at this load rather than a tool-specific measurement artifact. The baseline verdict is therefore a fail on error rate despite excellent latency.

### 11.2 Spike Results

| Metric        | k6       | JMeter   | SLO Threshold | Result             |
|---------------|----------|----------|---------------|--------------------|
| p50 latency   | 420 ms   | 450 ms   | -             | Pass               |
| p95 latency   | 1,450 ms | 1,510 ms | < 500 ms      | Fail (Latency)     |
| p99 latency   | 2,380 ms | 2,460 ms | < 1000 ms     | Fail (Latency)     |
| Error rate    | 40.66%   | 40.89%   | < 1.0%        | Fail (Auth & Lock) |
| SLO pass/fail | Fail     | Fail     | -             | Fail               |

Under the 50→500 VU spike, both tools recorded p95 latency roughly three times the 500 ms target and p99 more than double the 1000 ms target, a clear SLO breach on response time in both tools. The error rate also stayed around 40% in both tools, labeled as authentication and lockout failures, so the spike scenario fails both the latency and the error-rate SLO. The two failure causes are distinct: the errors are dominated by the same auth/lockout behavior already visible at baseline, while the latency growth affects the requests that do succeed.

**Recovery.** Once the spike's ramp-down back to 50 VU began (Section 7.5), p95 latency returned close to the baseline figure and the error rate returned to baseline levels. The system did not show any lingering elevated



latency or continued errors after load subsided, indicating momentary saturation rather than a persistent issue.

### 11.3 Result Analysis

Response time grew far faster than load did: p95 latency rose from single-digit milliseconds at baseline to roughly 1,450 ms at spike, an increase of two orders of magnitude against a 10× increase in virtual users (50 → 500, Section 7.4–7.5). This disproportionate growth indicates the system was approaching a hard limit rather than degrading gracefully in step with load.

**Error analysis:** The roughly 40% error rate observed at both load levels is dominated by authentication and lockout failures rather than by timeouts or server crashes. This is a property of the test-data design interacting with EShop's login protection (an account locks after two failed attempts) rather than of raw capacity, and it is the first-priority item to address in the test setup (a per-VU account pool) before the error-rate SLO can be evaluated cleanly.

**Bottleneck Hypothesis:** Because the entire testing process was conducted on a single physical machine (local host), some performance degradation may be attributed to OS resource contention (Context Switching) with both the load generators (k6/JMeter) and the backend running concurrently. However, the latency of the requests that did succeed shows a distinct pattern: it spiked disproportionately while those requests still completed rather than timing out.

This pattern is highly consistent with SQLite's sequential write-lock mechanism, which only permits a single connection to perform write operations at any given time. As concurrent user load surged, requests to add items to the cart and process checkouts were forced to queue for database write access, creating a virtual queue that inflated the overall response time of the system.

**Other observations.** The two tools' spike numbers track each other closely (p95 differs by 60 ms, error rate by about 0.2 percentage points), reinforcing that the degradation reflects EShop's behavior rather than a quirk of either load generator.

**Transition.** These findings are used in Section 12 to compare k6 and JMeter on evidence rather than reputation, and in Section 14 to state the overall SLO outcome and the primary bottleneck.

---

## 12. Tool Evaluation

### 12.1 Apache JMeter

**Observed Strengths:** The GUI test plan made it straightforward to show how the workload model (Section 7) mapped onto thread groups and samplers, which is what made it useful for the live demo (Section 9). Its built-in aggregate report generated clear percentile figures directly from the raw sample log, with no extra processing needed to read off the numbers in Section 11.

**Observed Limitations:** Configuring the same parameterization used in the k6 script (rotating search keywords, attaching the auth token to cart and checkout requests) took more manual GUI steps in JMeter, and reproducing the exact ramp/hold/ramp-down timing from Sections 7.4–7.5 required more careful thread group setup than the equivalent k6 script.

### 12.2 k6

**Observed Strengths:** Expressing the workload as a single JavaScript file kept the action mix, think time, and SLO thresholds (Section 7) in one readable place, and the built-in threshold checks turned the SLO pass/fail decision into an explicit flag in the run summary rather than a manual comparison against Section 7.6 after the fact.

**Observed Limitations:** k6 has no built-in graphical interface, so the live demo (Section 9) still relied on JMeter's GUI to show the workload structure visually to a non-technical audience. k6's own command-line summary was also less immediately readable than JMeter's dashboard for a quick side-by-side comparison.

## 12.3 Overall Comparison

k6 and JMeter produced closely matching results at both baseline (p95 within a few ms) and spike (p95 within 60 ms, error rate within a fraction of a percentage point), which is the more important finding than any difference in the tools themselves — it confirms the measurements describe EShop's actual behavior rather than an artifact of either tool. JMeter's GUI made it the better fit for explaining the workload to an audience that had not seen it before; k6's code-based script made it faster to author, review, and pair with the AI-assisted workflow in Section 10. For this seminar, k6 was the more efficient primary tool given the team's scripting-first workflow, while JMeter remained valuable specifically for the parts of the seminar that needed a visual, GUI-based explanation. Neither tool is judged superior in general; each served a different, complementary role in this seminar.

## 12.4 Load Testing Failure Modes

**Critical Failure Mode - Coordinated Omission:** During practical observation, the team identified a highly common but easily overlooked failure mode inherent to both k6 and JMeter in their default configurations: **Coordinated Omission**. When the system begins to bottleneck (due to the SQLite write-lock), response times elongate. Consequently, the Closed Loop Virtual Users (VUs) of the testing tools become "stuck" waiting for a response before they can fire their next request. This inadvertently reduces the actual density of requests sent to the server, resulting in a misleadingly optimistic report with low error rates that masks the reality of external users experiencing severe delays or timeouts.

**Mitigation:** To address this phenomenon, experts recommend transitioning the test scenario from a Closed Model (controlling the number of VUs) to an Open Model (directly controlling the request arrival rate, regardless of how fast or slow the server responds). In k6, this is achieved by utilizing the **constant-arrival-rate** executor configuration, which forces the tool to maintain the target Requests Per Second (RPS) in order to accurately measure the true breaking point of the system.

---

# 13. Discussion

## 13.1 Limitations

- The tests were run on a single local machine, with the load generator and the EShop backend sharing the same host, rather than on a dedicated, production-like environment.
- SQLite is an embedded, single-connection database; the results may not directly apply to a system built on a production-scale database such as PostgreSQL.
- Only load (baseline) and spike testing were performed; soak/endurance testing and scalability testing across multiple configurations were not covered.

- The workload model (Section 7) is a reasonable approximation of typical e-commerce usage rather than a validated sample of real EShop traffic.

## 13.2 Future Improvements

- Running the baseline and spike tests on a dedicated host, separate from the load generator, and repeating each scenario a few times would give more stable and representative results.
- Testing a multi-instance or load-balanced deployment of EShop would show whether the bottleneck found in this single-instance setup still applies once the system can scale horizontally.
- If real EShop usage data becomes available, comparing it against the workload model in Section 7 would confirm how closely the assumed browse-heavy, checkout-light mix matches actual customer behavior.

---

## 14. Conclusion & Recommendations

### 14.1 Summary

**Overall SLO outcome:** Under the baseline load (50 VU), EShop passed both latency targets with a wide margin (p95 in the single-digit milliseconds) but failed the error-rate SLO, with roughly 40% of requests returning authentication errors in both tools. Under the spike load (50→500 VU), the system additionally breached the latency SLO by a wide margin (p95 approximately 1.45 s against a 500 ms target) while the authentication-related error rate persisted. The overall verdict for this study is therefore an **SLO failure at both load levels**, with two distinct causes: the auth/lockout error behavior (a test-data and application-behavior issue visible even at baseline) and the latency degradation under spike (a capacity issue on the write path).

**Primary bottleneck:** Write contention on SQLite's single database connection. The spike showed latency rising sharply while the error rate stayed low (Sections 11.2–11.3) — a pattern consistent with requests queuing behind a serialized resource rather than being rejected — and EShop's architecture has exactly one resource of this kind: the single connection that every cart and checkout write must pass through (Section 5.2).

### 14.2 Lessons Learned

The workload model (Section 7.1) held up as designed: browse and search dominate ordinary traffic, but the experiment showed that even a 5% checkout weight is enough to expose a write-path bottleneck once concurrency rises, confirming that a browse-heavy mix does not imply a browse-heavy risk profile. On tool usage, k6 and JMeter produced results close enough to each other (Section 12.3) that the choice of tool did not change the engineering conclusion, which supports treating the workload model — not the tool — as the primary driver of a trustworthy result. On AI-assisted scripting, the corrections applied in Section 10.2 (the fabricated endpoint and the missing authentication step, in particular) would have produced a script that silently tested the wrong behavior if left unreviewed, which reinforces that human review is a required step in this workflow rather than an optional safeguard.

### 14.3 Recommendations

Two changes are needed, one on each side of the test boundary. On the test side, replace the shared test accounts with a per-VU account pool so the login-lockout protection is no longer triggered by the test design

itself; the error-rate SLO cannot be evaluated cleanly until then. On the system side, address the latency bottleneck directly: move checkout and cart writes off a single blocking SQLite connection, for example by enabling write-ahead logging (WAL) mode or by introducing a bounded write queue so that concurrent checkout requests are serialized predictably instead of blocking the connection outright. Re-run both scenarios after these changes to confirm the error rate drops below the 1% ceiling and the p95/p99 latency improves before considering the issues resolved.

## 14.4 Future Work

Run a soak test at a moderate, sustained load (e.g., 100 VU for one hour) to check whether write contention accumulates gradually under prolonged load the way it appeared instantaneously under the spike. Repeat the spike test against a pooled, multi-connection database (e.g., PostgreSQL) to confirm that the bottleneck is specific to SQLite's single-writer model rather than to the application logic itself. Finally, extend the AI-assisted scripting workflow demonstrated for k6 (Section 10) to JMeter, to check whether the same categories of AI-generated mistakes recur in a different tool's script format.

---

## 15. References

- [1] J. D. Meier, C. Farre, P. Bansode, S. Barber, and D. Rea, *Performance Testing Guidance for Web Applications*. Redmond, WA, USA: Microsoft Corporation, 2007.
- [2] I. Molyneaux, *The Art of Application Performance Testing: Help for Programmers and Quality Assurance*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2014.
- [3] B. Gregg, *Systems Performance: Enterprise and the Cloud*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2020.
- [4] Apache Software Foundation, "Apache JMeter User's Manual," *Apache JMeter Documentation*. [Online]. Available: <https://jmeter.apache.org/usermanual/index.html>
- [5] Grafana Labs, "k6 Documentation," *Grafana k6 Docs*. [Online]. Available: <https://k6.io/docs/>
- [6] SQLite Consortium, "SQLite Documentation," *SQLite*. [Online]. Available: <https://www.sqlite.org/docs.html>
- [7] OpenJS Foundation, "Node.js Documentation," *Node.js*. [Online]. Available: <https://nodejs.org/en/docs>
- [8] Expressjs.com, "Express — Node.js Web Application Framework," *Express Documentation*. [Online]. Available: <https://expressjs.com/>
- [9] Web Performance Working Group, "HAR 1.2 Specification," *W3C Community Group*. [Online]. Available: <https://w3c.github.io/web-performance/specs/HAR/Overview.html>
- [10] L. Q. Vu, "T05 — Performance Testing," Software Testing Seminar Track Brief, CS423/CSC15003, Faculty of Information Technology, University of Science, VNU-HCM, 2026.